



DMujeres Tracking

PLAN MAESTRO DE RECONSTRUCCIÓN, MIGRACIÓN Y NUEVA PLATAFORMA

Objetivo: pasar de la instalación actual en /DMujeres-Tracking a una instalación limpia en /home/DMujeres-Tracking, preservando usuarios, dispositivos, datos históricos y compatibilidad de las aplicaciones.

Decisión principal

La reconstrucción se hará en paralelo y la nueva plataforma será el producto. El sistema actual se conserva como fuente de datos, compatibilidad y respaldo durante la transición. El corte de producción solo ocurre después de backup verificable, restauración probada y pruebas E2E.

Documento preparado en español - septiembre de 2026

1. Resumen ejecutivo

La meta no es retocar el panel existente. La meta es crear una nueva plataforma DMujeres Tracking, con Web, App, API, servidor de tracking, almacenamiento y documentación propios, manteniendo el funcionamiento de los usuarios existentes.

El cambio de ubicación a /home/DMujeres-Tracking se trata como una migración de infraestructura, no como una instalación con datos vacíos. Se despliega código limpio, pero se recuperan datos reales mediante copias de seguridad y una estrategia de compatibilidad que conserva identificadores, credenciales, dispositivos, historiales y contratos necesarios para que las aplicaciones actuales sigan reportando.

Para producción, el stack recomendado es Node 24 LTS para servicios Web/API, React 19.3 + TypeScript + Vite 8.1 para la nueva Web, MapLibre GL JS 6.x para mapas, PostgreSQL 18.6 para datos, y una arquitectura donde el backend de Node no depende del esquema interno de almacenamiento del motor de tracking. Node 24 es LTS en septiembre de 2026; Node 20 está EOL. React 19.3 es la versión actual documentada oficialmente, Vite 8.1 es la rama estable publicada en junio de 2026 y PostgreSQL 18.6 es la versión 18 de mantenimiento publicada en agosto de 2026. [1][2][3][4][5]

2. Situación actual observada

El archivo recibido muestra una raíz con .github, backups, dashboard, dist, docs, fallback, infrastructure, mobile, server, variables de entorno y .gitmodules. También contiene una documentación técnica extensa sobre arquitectura, auditorías, seguridad, continuidad, OEM, rutas, replay, transporte y pruebas. Esto es valioso: no se debe tirar esa experiencia acumulada; se debe reutilizar como evidencia y convertirla en contratos y pruebas de la nueva plataforma.

Regla de migración

No usar una sustitución masiva de nombres ni mover carpetas productivas a ciegas. Primero inventario, luego backup, después reconstrucción, pruebas, corte y finalmente retiro del legado.

3. Arquitectura objetivo

ESTRUCTURA

```
DMujeres-Tracking/
├── apps/
│   ├── web/
│   └── mobile/
├── services/
│   ├── api/
│   └── tracking/
├── packages/
│   ├── contracts/
│   ├── shared-types/
│   └── config/
├── database/
│   ├── migrations/
│   ├── schema/
│   ├── indexes/
│   └── seeds/
└── infrastructure/
```

```
├── development/
├── production/
├── backup/

├── backups/
│   ├── legacy/
│   └── migration/

├── docs/
│   ├── architecture/
│   ├── api/
│   ├── app/
│   ├── database/
│   ├── security/
│   ├── deployment/
│   ├── operations/
│   └── migration/

├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   └── smoke/

├── scripts/
│   ├── backup/
│   ├── migration/
│   ├── validation/
│   └── release/

├── package.json
├── README.md
└── .env.example
```

La carpeta fuente actual /DMujeres-Tracking no debe ser destruida. La nueva instalación se crea en /home/DMujeres-Tracking. Durante el corte, el directorio viejo se conserva en modo de respaldo y solo se vuelve a él si la validación de recuperación lo exige.

4. Componentes y responsabilidades

Componente	Responsabilidad	Regla
apps/web	Panel completo para operación y administración	No conoce estructuras internas del servidor; usa /api/v1.
apps/mobile	Aplicación que captura y transmite posición/telemetría	Compatibilidad primero; evolución después.
services/api	Sesión, permisos, contratos, BFF, agregación y fachada	Node 24 LTS; código pequeño y claro.
services/tracking	Recepción, procesamiento, posiciones, eventos y persistencia	Se migra como servicio estable; no se rompe durante el primer corte.
database	Esquema, índices, particiones y migraciones	Versionado y reversible; no SQL manual sin migración.
infrastructure	systemd, Nginx, variables y despliegue	Separado por entorno.

5. API propia

```
RUTAS BASE
/api/v1/auth/login
```

```
/api/v1/auth/logout
/api/v1/auth/me

/api/v1/fleet
/api/v1/fleet/:id
/api/v1/fleet/:id/position

/api/v1/positions/live

/api/v1/replay
/api/v1/replay/:deviceId

/api/v1/reports/trips
/api/v1/reports/stops
/api/v1/reports/summary

/api/v1/battery
/api/v1/battery/:deviceId

/api/v1/users
/api/v1/users/:id

/api/v1/config
/api/v1/health
/api/v1/ready
/api/v1/version
```

Los contratos se diseñan para DMujeres Tracking. La Web no debe llamar rutas internas del servidor ni depender de nombres de tablas. Se crean DTOs y adaptadores que traducen la información al modelo de negocio de DMujeres Tracking.

6. Autenticación y compatibilidad con usuarios existentes

- Conservar los identificadores de usuario y las credenciales existentes durante la migración.
- No obligar a cambiar contraseñas solo por el cambio de servidor.
- Auditar el algoritmo de hash actual antes de cualquier mejora; si se puede, verificar con el formato existente y actualizar el hash después de un inicio de sesión exitoso.
- Mantener los mismos identificadores y credenciales de dispositivo utilizados por las aplicaciones actuales.
- Conservar dominio, hostname y rutas públicas cuando sea posible para que la App no necesite una actualización solo por el cambio de ubicación física del servidor.
- La nueva sesión Web debe utilizar una cookie propia HttpOnly, Secure y SameSite; la credencial interna del servidor no debe llegar al navegador.

7. PostgreSQL: diseño de almacenamiento para DMujeres Tracking

PostgreSQL 18.6 es una base sólida para el nuevo despliegue. La versión 18 incorpora mejoras de rendimiento, incluyendo AIO y otras optimizaciones; también introduce `uuidv7()`. Para este proyecto conviene combinar compatibilidad de IDs existentes con identificadores públicos ordenables para datos nuevos. [4][5]

El principio de diseño es separar el dato operativo de alta frecuencia del dato administrativo. No usar JSONB como sustituto del modelo relacional. La telemetría flexible puede vivir en JSONB, pero usuario, dispositivo, posición, batería, jornada y auditoría deben tener columnas e índices propios.

SQL - ESQUEMAS

```

CREATE SCHEMA IF NOT EXISTS iam;
CREATE SCHEMA IF NOT EXISTS tracking;
CREATE SCHEMA IF NOT EXISTS telemetry;
CREATE SCHEMA IF NOT EXISTS operations;
CREATE SCHEMA IF NOT EXISTS audit;
CREATE SCHEMA IF NOT EXISTS system;

CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
-- PostGIS es opcional: usarlo si se requiere geocerca/consultas espaciales.
-- CREATE EXTENSION IF NOT EXISTS postgis;

```

8. Modelo relacional recomendado

Dominio	Tablas principales	Motivo
iam	user_account, role, user_role, session	Identidad y permisos.
tracking	device, position, current_position, tracking_event	Operación y ubicación.
telemetry	battery_sample, signal_sample, device_health	Mediciones de alta frecuencia.
operations	journey, journey_segment, assignment, alert	Procesos y operación.
audit	audit_log	Trazabilidad administrativa.
system	setting, migration_map, schema_version	Configuración y migración.

SQL - NÚCLEO

```

CREATE TABLE iam.user_account (
  id          BIGINT PRIMARY KEY,
  public_id   UUID NOT NULL DEFAULT uuidv7() UNIQUE,
  username    TEXT NOT NULL UNIQUE,
  email       TEXT UNIQUE,
  password_hash TEXT NOT NULL,
  enabled     BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE tracking.device (
  id          BIGINT PRIMARY KEY,
  public_id   UUID NOT NULL DEFAULT uuidv7() UNIQUE,
  name        TEXT NOT NULL,
  unique_id   TEXT NOT NULL UNIQUE,
  enabled     BOOLEAN NOT NULL DEFAULT TRUE,
  status      TEXT NOT NULL DEFAULT 'UNKNOWN',
  last_seen_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE tracking.current_position (
  device_id   BIGINT PRIMARY KEY REFERENCES tracking.device(id),
  latitude    DOUBLE PRECISION NOT NULL,
  longitude   DOUBLE PRECISION NOT NULL,
  altitude_m  DOUBLE PRECISION,
  speed_kmh   REAL,
  course_deg  REAL,
  accuracy_m  REAL,
  battery_pct REAL,
  recorded_at TIMESTAMPTZ NOT NULL,

```

```

    received_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE tracking.position (
    id             BIGINT NOT NULL,
    device_id      BIGINT NOT NULL REFERENCES tracking.device(id),
    latitude       DOUBLE PRECISION NOT NULL,
    longitude      DOUBLE PRECISION NOT NULL,
    altitude_m     DOUBLE PRECISION,
    speed_kmh      REAL,
    course_deg     REAL,
    accuracy_m     REAL,
    recorded_at    TIMESTAMPTZ NOT NULL,
    received_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    attributes     JSONB,
    PRIMARY KEY (id, recorded_at)
) PARTITION BY RANGE (recorded_at);

```

SQL - PARTICIONES E ÍNDICES

```

CREATE TABLE tracking.position_2026_09
PARTITION OF tracking.position
FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');

CREATE INDEX position_2026_09_device_time_idx
ON tracking.position_2026_09 (device_id, recorded_at DESC);

CREATE INDEX position_2026_09_time_brin_idx
ON tracking.position_2026_09 USING BRIN (recorded_at);

CREATE INDEX position_2026_09_device_only_idx
ON tracking.position_2026_09 (device_id);

CREATE INDEX device_last_seen_idx
ON tracking.device (last_seen_at DESC);

CREATE INDEX event_device_time_idx
ON tracking.tracking_event (device_id, occurred_at DESC);

CREATE INDEX battery_device_time_idx
ON telemetry.battery_sample (device_id, recorded_at DESC);

```

La tabla position es de escritura intensiva y debe particionarse por tiempo. Un índice B-tree por (device_id, recorded_at DESC) acelera historial y replay; BRIN sobre recorded_at reduce el tamaño de índice para secuencias temporales grandes. current_position mantiene una fila por dispositivo para el panel en vivo, evitando ejecutar MAX(recorded_at) sobre millones de filas cada cinco segundos.

9. Reglas de rendimiento PostgreSQL

- No fijar shared_buffers, work_mem, effective_cache_size ni max_connections sin medir RAM, CPU, almacenamiento y concurrencia del VPS. Generar una configuración basada en recursos reales.
- Activar pg_stat_statements para detectar consultas caras y revisar el plan con EXPLAIN (ANALYZE, BUFFERS) en consultas críticas.
- Configurar autovacuum de forma específica para tablas de alta escritura; posición y telemetría no deben compartir una configuración genérica con tablas pequeñas.
- Crear particiones mensuales de forma anticipada y validar que no queden consultas que fuercen un escaneo de todas las particiones.

- Mantener JSONB solo para atributos variables; los campos usados para filtrar, ordenar o unir deben ser columnas normales e indexadas.
- Separar el almacenamiento de datos históricos de los datos de estado actual. El dashboard usa `current_position`; replay usa `position` particionada.
- Hacer backup y restauración de prueba, no solo dump. Un backup no verificado no es un respaldo operativo.

10. Migración de datos sin perder usuarios ni App

La frase "arrancar de 0" debe entenderse como "despliegue limpio de código e infraestructura", no como "base de datos vacía". Los datos actuales son parte del producto y deben entrar al nuevo entorno mediante copia controlada.

SHELL - MIGRACIÓN SEGURA

```
# 1) Preflight: identificar servicio, DB, usuario y configuración reales
sudo systemctl list-units --type=service | grep -Ei 'dm|track|java|postgres'
sudo ss -lntup
sudo -u postgres psql -c "\l"

# 2) Crear directorio de respaldos fuera del árbol de la aplicación
sudo mkdir -p /home/DMujeres-backups/migration

# 3) Dump lógico en formato custom (el nombre real se descubre en preflight)
sudo -u postgres pg_dump -Fc "$DB_NAME" -f "/home/DMujeres-backups/migration/${DB_NAME}_pre_cutover.dump"

# 4) Roles globales
sudo -u postgres pg_dumpall --globals-only > /home/DMujeres-backups/migration/postgres_globals.sql

# 5) Integridad del backup
sha256sum /home/DMujeres-backups/migration/* > /home/DMujeres-backups/migration/SHA256SUMS

# 6) El servidor actual se detiene SÓLO después de verificar los respaldos.
# 7) Crear la nueva instalación en /home/DMujeres-Tracking.
# 8) Restaurar la base de datos en el entorno nuevo.
# 9) Ejecutar pruebas de integridad y conteos antes de levantar el servicio.
```

Punto crítico

Nunca hardcodear `DB_NAME`, `DB_USER`, puertos ni nombres de servicios en los scripts de migración. El agente debe descubrirlos desde el entorno actual y registrar los valores usados en el runbook, sin copiar secretos al repositorio.

11. Validaciones de datos después de restaurar

SQL - VALIDACIÓN

```
-- Conteos base: comparar OLD vs NEW
SELECT count(*) FROM iam.user_account;
SELECT count(*) FROM tracking.device;
SELECT count(*) FROM tracking.position;

-- Unicidad de identificadores de dispositivos
SELECT unique_id, count(*)
FROM tracking.device
GROUP BY unique_id
HAVING count(*) > 1;
```

```
-- Última posición por dispositivo
SELECT device_id, max(recorded_at)
FROM tracking.position
GROUP BY device_id;
```

La comparación final debe incluir usuarios habilitados, dispositivos, identificadores únicos, última posición por dispositivo, rango de fechas del histórico, tokens necesarios para la App, configuraciones y cualquier dato operativo que el inventario determine como imprescindible.

12. Web nueva: experiencia y arquitectura

La Web se construye desde cero. React 19.3 + TypeScript + Vite 8.1 permiten mantener el desarrollo rápido y un código fácil de organizar. Vite 8 consolidó su empaquetado con Rolldown y la rama 8.1 añade mejoras para proyectos grandes. [2][3]

Área	Comportamiento esperado
Inicio	Resumen operativo, unidades en línea, alertas, batería baja y estado del sistema.
En vivo	Mapa + lista + detalle, filtros y actualización incremental.
Historial	Selección de dispositivo, fecha y rango de tiempo.
Replay	Reproducción fluida, controles temporales y trazado segmentado.
Batería	Estado actual, tendencia y alertas.
Reportes	Viajes, paradas, actividad y resumen.
Usuarios	Gestión de acceso y roles.
Configuración	Preferencias del sistema y operación.

13. Replay de nueva generación

El replay debe ser una de las áreas donde más se note el salto de calidad. No debe pintar cada punto como un elemento independiente del DOM. El recorrido se representa como geometría, los eventos como capas separadas y el marcador actual como un único objeto animado.

- Carga por ventana temporal, no todo el histórico de una vez.
- Simplificación geométrica configurable según zoom y densidad.
- Colores o estados de segmento por movimiento, pausa, pérdida de señal o evento.
- Marcador actual con hora, velocidad y batería.
- Velocidades 0.5x, 1x, 2x, 4x, 8x y 16x.
- Botones para siguiente evento, salto temporal y volver a la posición actual.
- Cache de puntos ya procesados para no recalcular al retroceder.
- Desacoplamiento entre reproducción temporal y renderizado del mapa.

14. App: continuidad antes que rediseño

La App actual tiene lógica propia de seguimiento, continuidad, diagnóstico, salud del dispositivo y recuperación. Esa lógica debe inventariarse y convertirse en requisitos. La nueva App se construye después de estabilizar la API y la compatibilidad del servidor.

- La migración del servidor debe conservar el contrato utilizado por la App actual.
- Conservar identificación de dispositivo y credenciales.
- Probar arranque, login/configuración, permisos, ubicación, envío, reintentos, recuperación y funcionamiento con pantalla apagada.
- Liberar la nueva App solo cuando el servidor nuevo haya pasado las pruebas de compatibilidad.

15. Infraestructura y corte de producción

ARQUITECTURA DE DESPLIEGUE



El servidor nuevo vive en /home/DMujeres-Tracking. Los puertos internos deben estar limitados por firewall y, en lo posible, enlazados a 127.0.0.1 o a una red privada. Nginx expone solo HTTPS. La App conserva el endpoint público actual mientras exista compatibilidad.

16. Orquestación de subagentes

La IA debe operar como un equipo coordinado y no como varios agentes escribiendo sobre la misma carpeta sin control. El Orquestador decide orden, dependencias, conflictos y puntos de corte. Los especialistas trabajan en paralelo cuando no comparten archivos críticos.

Rol	Responsabilidad	Puede modificar producción
Orquestador	Plan, dependencias, revisión final y estado	No, salvo cutover autorizado.
Auditor	Inventario del sistema actual y compatibilidad	No
DB	PostgreSQL, migraciones, índices y restauración	No
Backend	API, sesión, adapters, health y rendimiento	No
Web	React, mapa, replay y UX	No
App	compatibilidad y nueva App	No
Infra	systemd, Nginx, firewall y despliegue	Solo mediante checkpoint
QA	tests, E2E, smoke, regresión	No
Seguridad	sesiones, secretos y superficies de ataque	No
Release	cutover y rollback	Sí, solo tras todos los gates

CONTRATO DE TRABAJO ENTRE AGENTES



Cada agente entrega:

1. cambios
2. pruebas
3. riesgos
4. archivos tocados
5. siguiente dependencia

17. Gates antes del corte

1. Backup de base de datos completo, verificable por SHA-256 y restaurado en un entorno de prueba.
2. Backup de configuración, certificados y secretos operativos almacenado fuera del directorio del proyecto.
3. Inventario de usuarios, dispositivos, endpoints y puertos terminado.
4. Nueva instalación en /home/DMujeres-Tracking compilada y probada sin tocar producción.
5. Restauración de datos validada con conteos e identificadores.
6. App actual capaz de conectarse al servidor nuevo mediante el mismo endpoint público.
7. Web nueva funcional en login, mapa, posiciones, replay, batería, reportes y usuarios.
8. Pruebas E2E y smoke aprobadas.
9. Rollback ensayado y documentado.
10. Solo entonces: apagar temporalmente el servicio actual, hacer el corte y levantar el servicio nuevo.

18. PROMPT MAESTRO PARA LA IA ORQUESTADORA

El siguiente bloque está preparado para pegarlo directamente en un agente de programación. Está redactado para que el agente trabaje por fases, utilice subagentes, mantenga el sistema actual a salvo y ejecute el corte únicamente al final.

PROMPT MAESTRO

```
# PROYECTO: DMUJERES TRACKING - RECONSTRUCCIÓN COMPLETA EN /home
```

```
## OBJETIVO
```

Reconstruir la plataforma completa como un producto propio llamado "DMujeres Tracking" en:

```
/home/DMujeres-Tracking
```

La instalación actual se encuentra en:

```
/DMujeres-Tracking
```

El nuevo entorno debe ser limpio, ordenado, mantenible, escalable y preparado para Web + App + API + servidor + PostgreSQL + infraestructura.

La prioridad máxima es conservar usuarios, dispositivos, históricos y compatibilidad de las aplicaciones actuales.

"Arrancar desde 0" significa arrancar desde 0 en código e infraestructura, NO arrancar con datos vacíos.

```
=====
```

```
1. REGLAS ABSOLUTAS
```

```
=====
```

NO borrar /DMujeres-Tracking.
 NO reemplazarlo.
 NO modificarlo de forma destructiva.
 NO borrar usuarios.
 NO borrar dispositivos.
 NO borrar históricos.
 NO borrar credenciales.
 NO cambiar contraseñas de usuarios por obligación.
 NO romper la App actual.
 NO alterar producción antes del checkpoint de corte.
 NO ejecutar migraciones destructivas.
 NO hacer DROP DATABASE sobre producción.
 NO publicar secretos.
 NO hardcodear credenciales.
 NO afirmar que algo funciona sin probarlo.

Todo cambio peligroso debe tener:

- backup
- evidencia
- plan de rollback
- validación

2. MODO DE TRABAJO

Actúa como un equipo de ingeniería coordinado por un ORQUESTADOR.

No trabajes como un único agente improvisando.

Usa subagentes especializados:

- A. AUDITOR
- B. ARQUITECTURA
- C. BASE DE DATOS
- D. BACKEND/API
- E. WEB
- F. APP
- G. INFRAESTRUCTURA
- H. SEGURIDAD
- I. QA/E2E
- J. DOCUMENTACIÓN
- K. RELEASE/ROLLBACK

El ORQUESTADOR es responsable de:

- dividir tareas
- decidir dependencias
- evitar conflictos
- revisar resultados
- mantener un registro de estado
- impedir cambios prematuros en producción

Los subagentes pueden trabajar en paralelo cuando no dependan entre sí.

Ningún subagente puede detener producción por su cuenta.

Solo RELEASE puede ejecutar el corte y únicamente después de todos los gates.

3. PRIMERA FASE: INVENTARIO

Antes de crear código, inspeccionar:

- estructura del proyecto
- Git
- submódulos
- servicios systemd
- puertos
- Nginx
- firewall
- variables de entorno
- base de datos
- usuarios
- dispositivos
- API actuales
- endpoints usados por la App
- credenciales de dispositivos
- FCM
- MQTT
- TLS
- jobs
- backups
- scripts
- logs
- documentación
- pruebas existentes

Crear:

docs/agents/INVENTARIO.md

El documento debe incluir:

- estado actual
- servicios
- rutas
- puertos
- dependencias
- base de datos
- volumen aproximado de datos
- usuarios
- dispositivos
- riesgos
- compatibilidades necesarias

No copiar secretos.

4. BACKUP OBLIGATORIO

Antes de modificar o detener nada:

Crear:

/home/DMujeres-backups/migration/<fecha-hora>/

Respaldar:

- PostgreSQL
- roles globales
- configuración
- variables de entorno sin exponer secretos en Git
- certificados
- configuración Nginx
- servicios systemd
- scripts operativos
- configuración del servidor de tracking
- cualquier dato necesario para que la App continúe funcionando

Usar pg_dump en formato custom y pg_dumpall --globals-only.

Generar SHA-256 de todos los backups.

Después restaurar el dump en un entorno de prueba y comprobar que realmente abre.

NO considerar válido un backup que no haya sido restaurado al menos una vez.

5. NUEVA RAÍZ

Crear:

/home/DMujeres-Tracking

Si ya existe:

- inspeccionar
- no borrar
- detenerse si contiene trabajo no documentado

La estructura objetivo es:

```
DMujeres-Tracking/
├── apps/
│   ├── web/
│   └── mobile/
├── services/
│   ├── api/
│   └── tracking/
├── packages/
│   ├── contracts/
│   ├── shared-types/
│   └── config/
├── database/
│   ├── migrations/
│   ├── schema/
│   ├── indexes/
│   └── seeds/
├── infrastructure/
│   ├── development/
│   ├── production/
│   └── backup/
├── backups/
├── docs/
├── tests/
├── scripts/
└── README.md
```

```
└─ .env.example
```

6. IDENTIDAD

Toda la nueva interfaz, documentación de producto, nombres de módulos propios, logs y mensajes deben usar:

DMujeres Tracking

No usar la marca anterior en la experiencia del usuario.

No poner textos que hagan parecer el producto una copia.

No escribir comentarios artificiales.

No escribir frases como:

- "como IA"
- "solución óptima"
- "este código generado"
- "implementación inteligente"

Los comentarios deben explicar decisiones reales.

Los mensajes deben sonar escritos por un desarrollador que mantiene el sistema.

Importante: no eliminar avisos de copyright/licencia de terceros cuando legalmente deban conservarse.

7. STACK

Usar versiones estables y soportadas al momento de implementación.

Base objetivo:

- Node.js 24 LTS
- React 19.3
- TypeScript
- Vite 8.1
- MapLibre GL JS 6.x
- Chart.js
- TanStack Query
- Zustand
- PostgreSQL 18.6

Backend Node:

- node:http
- fetch nativo
- WebCrypto
- AbortController
- módulos ESM

Mantener el runtime del servicio Node lo más pequeño posible.

No introducir frameworks grandes sin una justificación concreta.

8. ARQUITECTURA

Separar:

WEB
API
CASOS DE USO
ADAPTADORES
SERVIDOR DE TRACKING
BASE DE DATOS

La Web nunca debe conocer los detalles internos del servidor.

Crear API propia bajo:

/api/v1/

9. API PROPIA

Crear, como mínimo:

POST /api/v1/auth/login
POST /api/v1/auth/logout
GET /api/v1/auth/me

GET /api/v1/fleet
GET /api/v1/fleet/:id
GET /api/v1/fleet/:id/position
GET /api/v1/positions/live

GET /api/v1/replay
GET /api/v1/replay/:deviceId

GET /api/v1/reports/trips
GET /api/v1/reports/stops
GET /api/v1/reports/summary

GET /api/v1/battery
GET /api/v1/battery/:deviceId

GET /api/v1/users
GET /api/v1/users/:id

GET /api/v1/config
GET /api/v1/health
GET /api/v1/ready
GET /api/v1/version

No devolver directamente estructuras internas del servidor.
Crear DTOs propios.

10. SESIONES

El navegador debe manejar una sesión propia de DMujeres Tracking.

Usar:
HttpOnly
Secure
SameSite

Las credenciales internas no salen al navegador.

Manejar:
- login
- logout
- expiración
- 401
- 403
- timeout
- permisos

11. COMPATIBILIDAD CON LA APP ACTUAL

Antes de cortar producción, descubrir exactamente qué utiliza la App.

Conservar:
- host público
- rutas necesarias
- identificadores de dispositivo
- credenciales de dispositivo
- configuración de transporte
- contratos mínimos de envío
- respuestas necesarias

La App actual debe poder reconectar al nuevo servidor sin necesitar una nueva versión solo por el cambio de directorio en el VPS.

Si una mejora exige cambiar el contrato, construir una capa de compatibilidad.

12. POSTGRESQL

Diseñar PostgreSQL como sistema de datos del producto y del servidor.

Schemas mínimos:

iam
tracking
telemetry
operations
audit
system

Usar:
- claves internas BIGINT cuando sea importante preservar compatibilidad
- public_id UUID ordenable para nuevas referencias públicas
- tablas temporales particionadas
- B-tree para device_id + tiempo
- BRIN para tiempo en tablas grandes

- JSONB solo para atributos variables

Crear tablas como:

```
iam.user_account
tracking.device
tracking.position
tracking.current_position
tracking.tracking_event
telemetry.battery_sample
telemetry.device_health
operations.journey
operations.journey_segment
operations.assignment
audit.audit_log
system.setting
system.migration_map
system.schema_version
```

La tabla tracking.position debe particionarse por recorded_at.

Mantener tracking.current_position con una fila por dispositivo para el panel en vivo.

=====

13. MIGRACIÓN DE USUARIOS Y DATOS

=====

La base nueva se crea limpia, pero se restaura el contenido real.

Conservar:

- ids de usuarios
- ids de dispositivos
- uniqueId
- credenciales
- históricos
- atributos necesarios
- tokens de recuperación
- configuraciones necesarias

No realizar un cambio de contraseña masivo.

Auditar el algoritmo de hash existente antes de modernizar autenticación.

No perder fechas históricas.

Verificar conteos OLD vs NEW.

=====

14. WEB

=====

Construir desde cero.

React + TypeScript + Vite.

Páginas mínimas:

- login

- inicio
- en vivo
- detalle de unidad
- historial
- replay
- batería
- reportes
- usuarios
- configuración
- estado del sistema

La aplicación debe ser rápida y clara.

No copiar el diseño anterior.

=====

15. MAPA

=====

Usar MapLibre GL JS.

Evitar recrear todos los marcadores cada pocos segundos.

Actualizar solo cambios.

Separar estado del mapa y estado de los datos.

Usar clustering cuando tenga sentido.

=====

16. REPLAY

=====

Diseñar un replay completamente nuevo.

Requisitos:

- selección de unidad
- fecha
- rango horario
- reproducción
- pausa
- velocidades 0.5x, 1x, 2x, 4x, 8x, 16x
- salto al siguiente evento
- trazado segmentado
- indicador actual
- velocidad
- batería
- dirección
- hora

Optimizar con:

- carga por ventanas
- simplificación geométrica
- cache
- renderizado eficiente
- procesamiento incremental

No usar miles de elementos DOM para el recorrido.

17. POLLING

Primera versión:
5 segundos aproximados.

Pero implementar:

- AbortController
- control de solapamiento
- retry razonable
- backoff ante error
- pausa cuando la pestaña no necesita actualizaciones
- recuperación automática

Preparar arquitectura para SSE/WebSocket sin romper API.

18. TESTS

Debe haber:

- unit
- integration
- smoke
- E2E

Comprobar:

- login
- logout
- sesión expirada
- permisos
- fleet
- posición
- replay
- batería
- reportes
- usuarios
- caída de servicio
- recuperación
- App actual conectando

19. INFRAESTRUCTURA

Crear servicios separados.

Ejemplo conceptual:

dmujeres-tracking.service
dmujeres-api.service
dmujeres-web.service

Usar Nginx delante.

No exponer puertos internos innecesarios.

20. CUTOVER

=====

SOL0 cuando todos los gates estén verdes:

1. poner ventana de mantenimiento
2. detener temporalmente el servicio actual
3. ejecutar backup final
4. verificar SHA-256
5. asegurar que no hay cambios nuevos
6. levantar base de datos nueva
7. levantar servidor nuevo
8. levantar API
9. levantar Web
10. ejecutar smoke test
11. probar login
12. probar usuario real controlado
13. probar dispositivo real
14. verificar llegada de posición
15. verificar batería
16. verificar replay
17. verificar reportes
18. verificar App actual
19. mantener monitorización intensiva

Si algo crítico falla:

ROLLBACK.

No improvisar sobre producción.

=====

21. ROLLBACK

=====

Documentar exactamente:

- cómo detener nuevo
- cómo volver a servicios anteriores
- cómo restaurar configuración
- cómo validar la BD anterior
- cómo reactivar usuarios
- cómo verificar que la App vuelve a reportar

=====

22. DOCUMENTACIÓN

=====

Todo en español.

Crear:

docs/architecture/
docs/api/
docs/app/
docs/database/
docs/security/
docs/deployment/
docs/operations/
docs/migration/

Crear ADRs para decisiones importantes.

23. CALIDAD

No crear archivos gigantes.
 No crear carpetas sin propósito.
 No duplicar lógica.
 No meter lógica de negocio en componentes React.
 No meter SQL arbitrario en controladores.
 No usar catches vacíos.
 No silenciar errores.
 No dejar TODOs como sustituto de trabajo terminado.

Cuando exista una duda real, inspeccionar el código y la infraestructura antes de decidir.

24. ENTREGA POR FASES

FASE 0 - Inventario
 FASE 1 - Backup
 FASE 2 - Arquitectura y contratos
 FASE 3 - PostgreSQL y restauración
 FASE 4 - Backend/API
 FASE 5 - Web
 FASE 6 - Replay
 FASE 7 - App
 FASE 8 - Infra
 FASE 9 - QA/E2E
 FASE 10 - Cutover
 FASE 11 - Estabilización
 FASE 12 - Retiro gradual del legado

No saltar fases críticas.

25. REPORTE OBLIGATORIO DEL ORQUESTADOR

Al terminar cada fase informar:

- qué se hizo
- qué archivos se crearon
- qué archivos se modificaron
- qué pruebas pasaron
- qué pruebas fallaron
- riesgos
- decisiones
- backup utilizado
- siguiente fase

Nunca decir "todo listo" si faltan pruebas.

26. DEFINICIÓN DE TERMINADO

```
[ ] nueva raíz /home/DMujeres-Tracking
[ ] inventario completo
[ ] backup restaurado
[ ] usuarios conservados
[ ] dispositivos conservados
[ ] históricos conservados
[ ] App actual compatible
[ ] API propia
[ ] nueva Web
[ ] nuevo replay
[ ] PostgreSQL organizado
[ ] índices y particiones
[ ] health/ready
[ ] systemd
[ ] Nginx
[ ] seguridad
[ ] tests
[ ] E2E
[ ] rollback probado
[ ] documentación
[ ] cutover probado
```

Solo después de marcar todo como verificado se puede considerar terminada la migración.

19. Instrucción operativa final para el agente

Regla de oro

La nueva plataforma puede ser nueva; los datos y la continuidad del negocio no. El código se reconstruye, los datos se migran, la compatibilidad se prueba y el corte se hace una sola vez y con rollback listo.

Durante el desarrollo, el agente debe preferir pequeñas decisiones verificables sobre grandes refactors simultáneos. Si encuentra un conflicto entre una mejora arquitectónica y la continuidad de usuarios, debe preservar continuidad y diseñar un adaptador o una fase de transición.

20. Checklist de corte en una sola página

Gate	Condición para continuar
Backup DB	Dump creado + SHA-256 + restauración de prueba.
Usuarios	Conteos y accesos válidos en el entorno nuevo.
Dispositivos	Identificadores y credenciales conservados.
App	Prueba real de conexión y envío.
API	Contrato /api/v1 documentado y probado.
Web	Login + mapa + replay + reportes + batería.
PostgreSQL	Índices/particiones y consultas críticas revisadas.
Infra	systemd + Nginx + firewall verificados.
Seguridad	Cookies/secretos/permisos revisados.
Rollback	Procedimiento ensayado.

21. Fuentes técnicas consultadas

[1] Node.js - Releases y soporte de ramas: Node 24 LTS; Node 20 EOL.

[2] React - documentación de versiones: React 19.3.

[3] Vite - anuncios oficiales: Vite 8.0 y Vite 8.1; Rolldown como bundler unificado.

[4] PostgreSQL - documentación y notas de versión 18.6.

[5] PostgreSQL 18 - novedades: AIO, uuidv7(), mejoras de índices y del planificador.

[6] MapLibre - boletines oficiales 2026: rama GL JS 6.x y mejoras de rendimiento.

[7] TanStack Query - documentación oficial: consultas y optimizaciones de renderizado.

- Node.js - documentación oficial (nodejs.org)
- React - documentación oficial (react.dev)
- Vite - documentación oficial (vite.dev)
- PostgreSQL - documentación oficial (postgresql.org)
- MapLibre - documentación oficial (maplibre.org)
- TanStack Query - documentación oficial (tanstack.com)

DMujeres Tracking - documento maestro de migración y reconstrucción